

พื้นฐานการวิเคราะห์และออกแบบระบบเชิงวัตถุด้วย *UML*

บทนำ

การมองปัญหาในภาพรวมด้วยการตัดรายละเอียดทิ้งไปเพื่อลดความซับซ้อนลง โดยเรียกเทคนิคนี้ว่า *Abstraction*

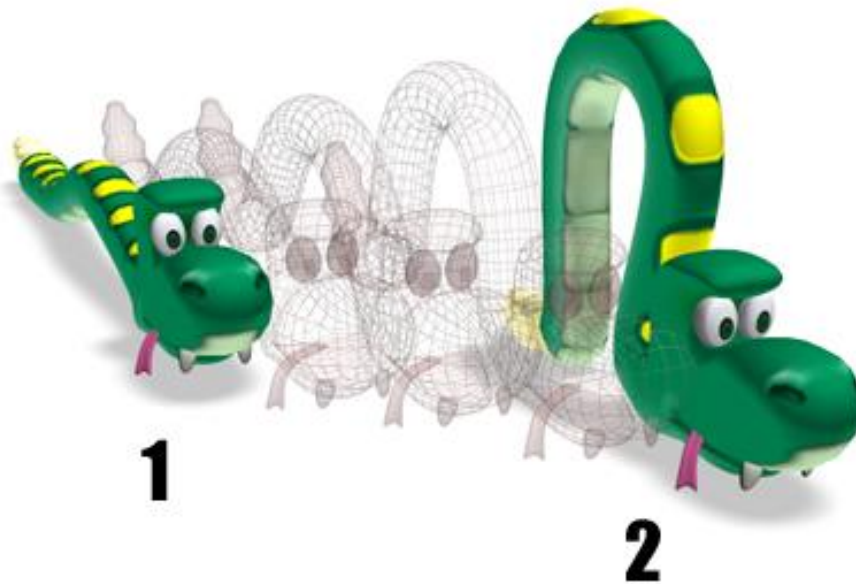
หัวข้อการเรียนรู้

1. ออบเจกต์ (Object), คลาส (Class), การสืบทอดคุณสมบัติ (Inheritance), โพลิมอร์ฟิซึม (Polymorphism), เอ็นแคปซูลेशन และการซ่อนรายละเอียด (Encapsulation and Information Hiding)
2. ความสัมพันธ์ของออบเจกต์ และการมีส่วนร่วม (Object Relationships and Associations)
3. หลักการพัฒนาระบบเชิงวัตถุ และ UML (Unified Modeling Language)



ออบเจกต์ (Object)

ออบเจกต์หรือวัตถุ คือหน่วยสนใจของระบบที่ทำให้เกิดเหตุการณ์บางอย่าง ไม่ว่าจะเป็นบุคคล สถานที่ สิ่งของ หรือเหตุการณ์ สำหรับออบเจกต์ในโลกของเทคโนโลยีเชิงวัตถุ จะเน้นที่ตัวปฏิบัติการมากกว่าการปฏิบัติ



คลาส (Class)

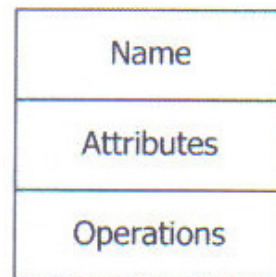
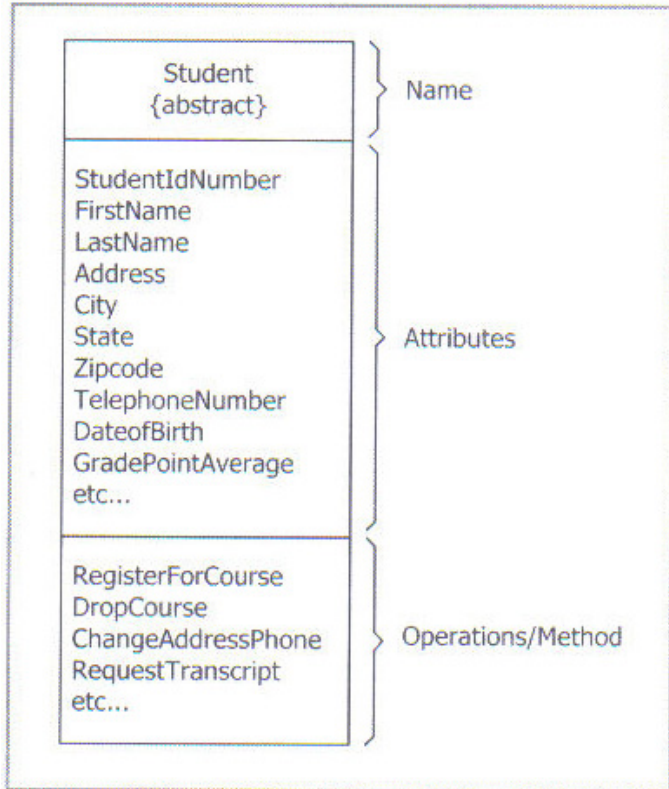
คลาสถือว่าเป็นนามธรรม (Abstract) ในขณะที่ออบเจกต์นั้นเป็นสิ่งที่มีความเป็นตัวตน (Concrete) คลาสก็คือแบบพิมพ์เขียวของออบเจกต์โดยที่คลาสไม่สามารถทำงานได้ ในขณะที่ออบเจกต์สามารถทำงานได้ การทำงานของออบเจกต์จะเป็นไปตามคุณสมบัติที่กำหนดไว้ในคลาส และออบเจกต์ทุกตัวก็ต้องอยู่ในคลาส โดยเราสามารถดูคุณสมบัติ (Characteristic) ของออบเจกต์ได้ด้วยการดูที่คลาส

คลาสคือกลุ่มของออบเจกต์ที่มีโครงสร้างพื้นฐานพฤติกรรมเดียวกัน ดังนั้นออบเจกต์ที่มีคุณสมบัตินี้ลักษณะเดียวกัน ก็จะรวมกลุ่มอยู่ในคลาสเดียวกัน จึงสรุปได้ว่าคลาสก็คือต้นแบบข้อมูล ที่มีไว้เพื่อสร้างออบเจกต์



คลาส (Class)

คลาส นอกจากจะมีชื่อคลาสดำกับแล้ว ยังมีแอตทริบิวต์ (Attribute) ที่ใช้อธิบายคุณสมบัติ และ โอเปอเรชัน (Operation) ที่ใช้อธิบายถึงพฤติกรรมของคลาสว่ามีตัวปฏิบัติการอะไรบ้าง ซึ่งโอเปอเรชันบางครั้งอาจเรียกว่าเมธอด (Method) ก็ได้ โดยทั้งคลาส แอตทริบิวต์ และ โอเปอเรชัน สามารถแสดงในลักษณะเพิ่มเติมได้



การสืบทอดคุณสมบัติ (Inheritance)

ข้อดี

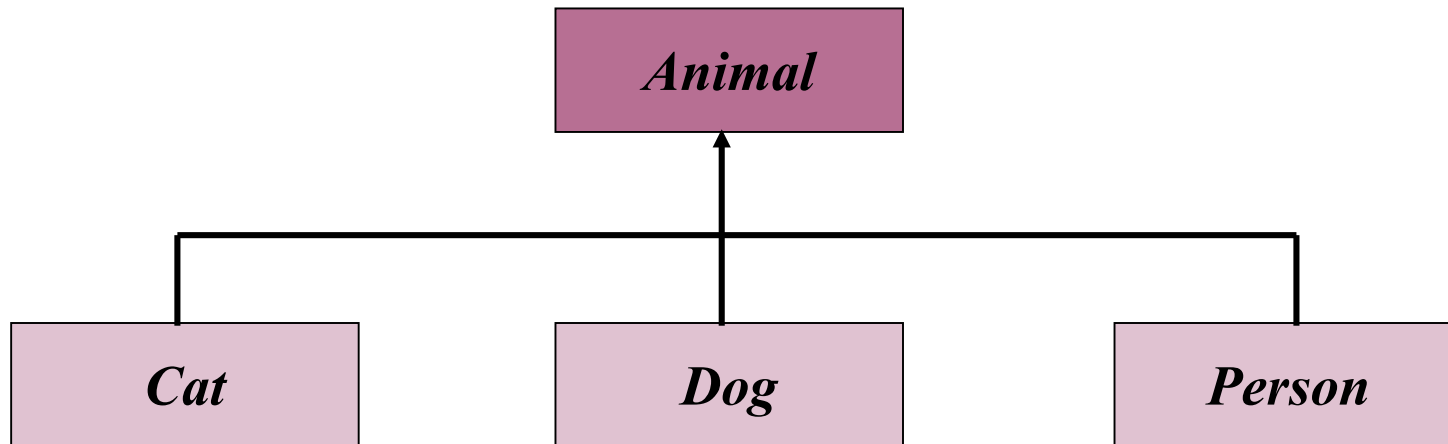
1. ทำให้มีโครงสร้างเป็นระบบ ระเบียบ ปรับเปลี่ยนได้ง่าย
2. ลดเวลาในการพัฒนาระบบ
3. ลดค่าใช้จ่ายในการพัฒนา



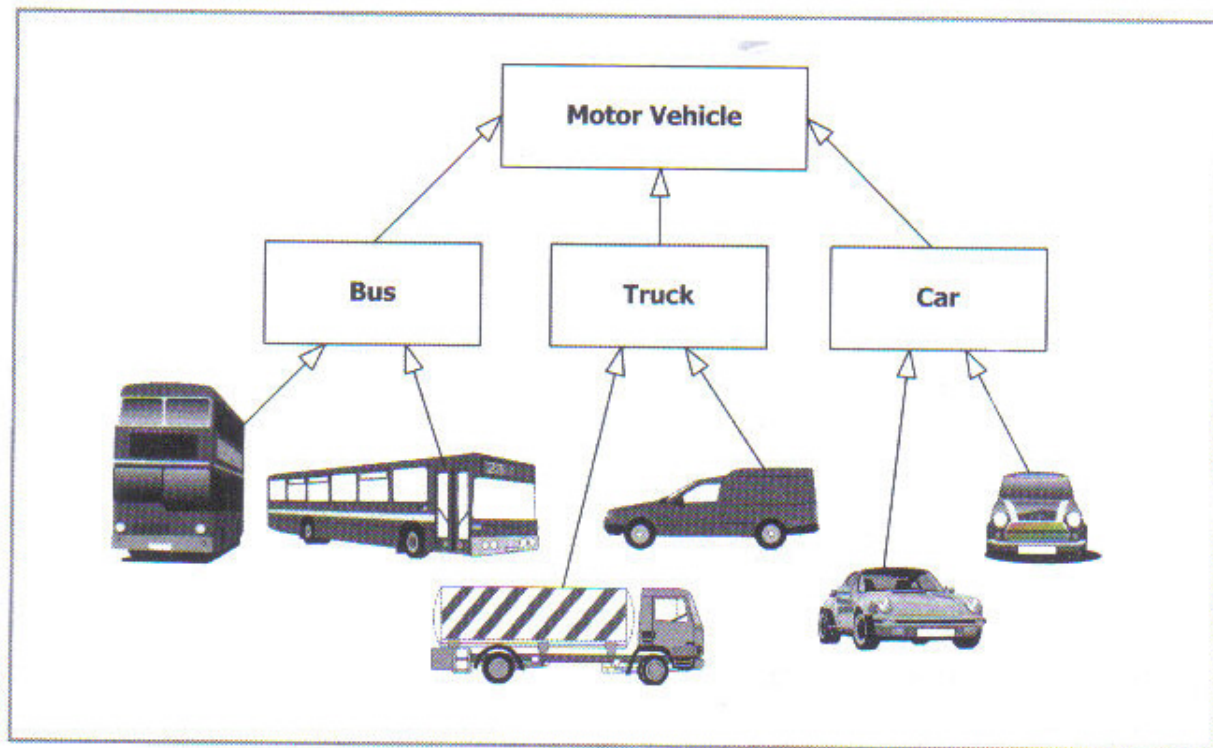
การกำหนดคุณสมบัติของออบเจกต์แต่ละตัวในระบบ จะใช้วิธีการสืบทอด (Inheritance) โดยอาศัยคุณสมบัติของออบเจกต์ที่มีอยู่แล้วใส่ลงในออบเจกต์ตัวใหม่ แนวความคิดเชิงวัตถุจะถือว่าการสืบทอดเป็นสิ่งสำคัญ เพราะว่าไม่มีสิ่งใดในโลกที่เกิดขึ้นเอง ต้องมีการสืบทอด



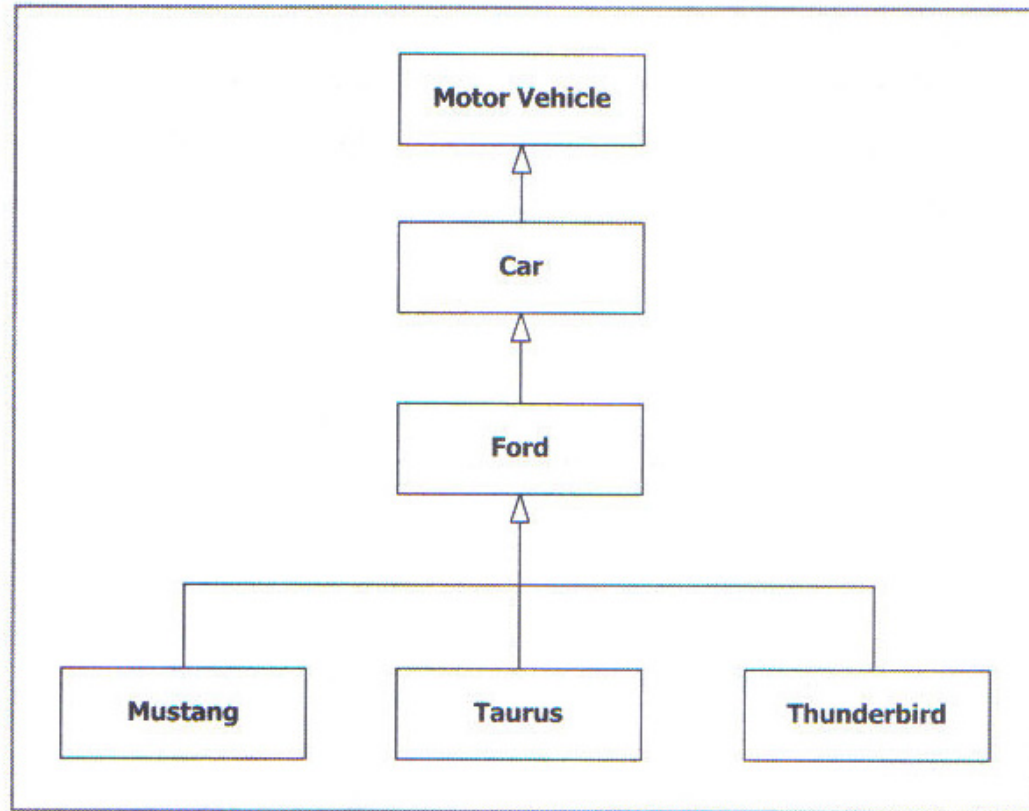
การสืบทอดคุณสมบัติ (Inheritance)



การสืบทอดคุณสมบัติ (Inheritance)

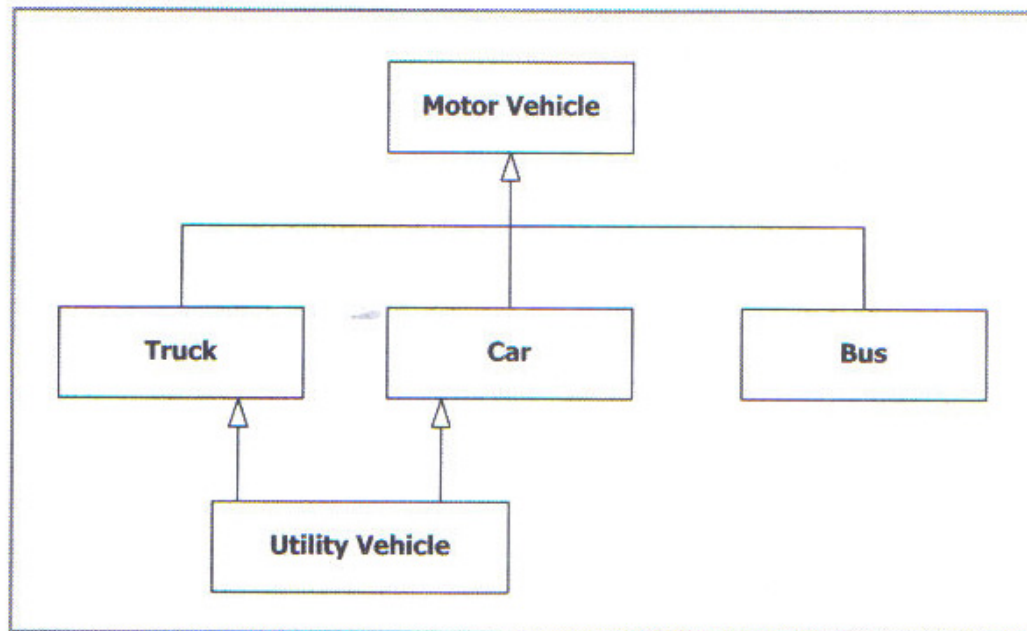


การสืบทอดคุณสมบัติ (Inheritance)



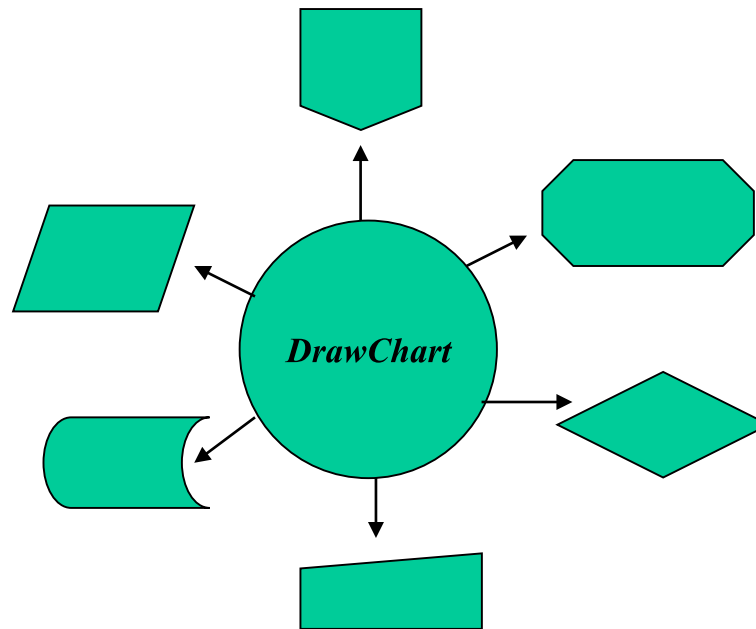
การสืบทอดคุณสมบัติ (Inheritance)

การสืบทอดคุณสมบัติของ Subclass ยังสามารถสืบทอดจาก Superclass มากกว่าหนึ่งก็ได้ ที่เรียกว่า *Multiple Inheritance*



โพลิมอร์ฟิซึม (Polymorphism)

โพลิมอร์ฟิซึม (Polymorphism) หมายถึง การบอกแบบเดียว แต่ได้รับการตอบสนองหลายรูปแบบ



คุณสมบัติของ *Polymorphism* ทำให้สามารถวาดรูปได้หลายรูปแบบ ด้วยการส่งผ่านฟังก์ชัน *DrawChart()* เพียงฟังก์ชันเดียว



เอ็นแคปซูลชัน และการซ่อนรายละเอียด (Encapsulation and Information Hiding)

การซ่อนรายละเอียด (Information Hiding) เป็นพื้นฐานของการปกปิดข้อมูลภายใน และวิธีการทำงานของออบเจกต์ โดยคำว่าเอ็นแคปซูลชัน มีความหมายว่าผู้ใช้ไม่สามารถ เห็นรายละเอียด (ข้อมูลและฟังก์ชัน) ภายใน เนื่องจากถูกซ่อนและบรรจุไว้ในแคปซูล แต่ สามารถใช้งานออบเจกต์ได้ด้วยเมธอด กล่าวคือ ในการเข้าถึงข้อมูลนั้น จะไม่สามารถ เข้าถึงได้โดยตรง ต้องมีการตอบรับจากเมธอดในออบเจกต์ปลายทางนั้นว่า จะอนุญาต หรือไม่ที่จะให้ออบเจกต์ที่ส่งเมสเสจร้องขอเข้าถึงข้อมูลตน

สำหรับใน UML การกำหนดให้การมองเห็น (Visibility) ของแอตทริบิวต์หรือ เมธอดว่าเป็น Public, Protected หรือ Private จะใช้สัญลักษณ์ดังต่อไปนี้

+ คือ สัญลักษณ์ Public Visibility

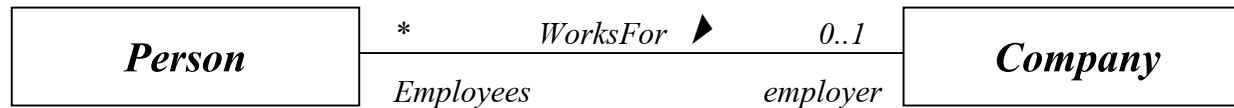
คือ สัญลักษณ์ Protected Visibility

- คือ สัญลักษณ์ Private Visibility

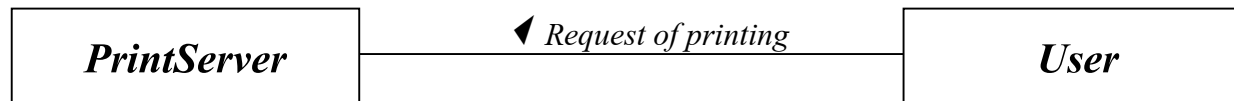


ความสัมพันธ์ของออบเจกต์ และการมีส่วนร่วม (Object Relationships and Associations)

1. *Association* คือ ความสัมพันธ์ระหว่างออบเจกต์



แสดงความสัมพันธ์ระหว่างพนักงาน และบริษัท ในรูปแบบ *Bidirectional Association*

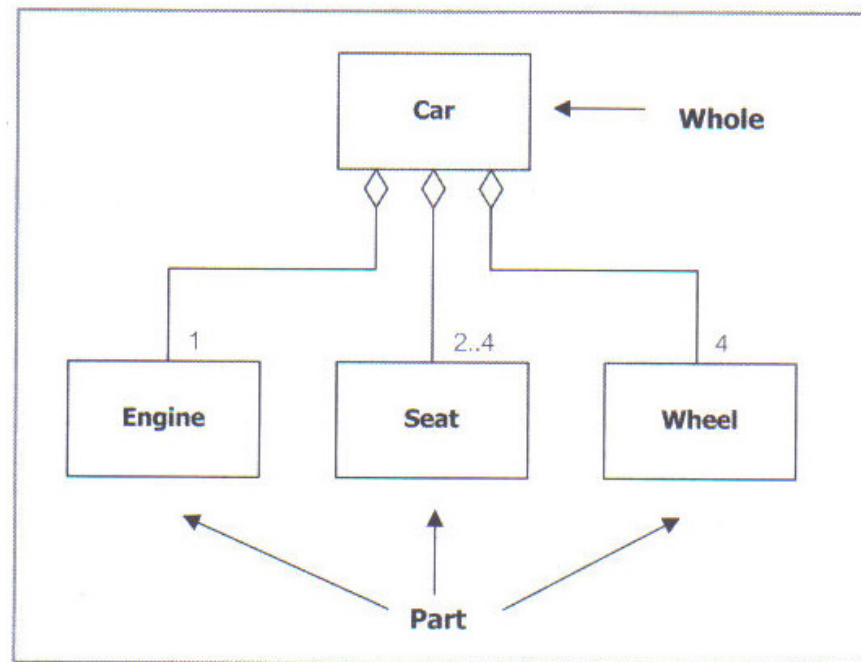


แสดงความสัมพันธ์ระหว่างผู้ใช้ และเครื่องพิมพ์ ในรูปแบบ *Consumer-Producer Association*



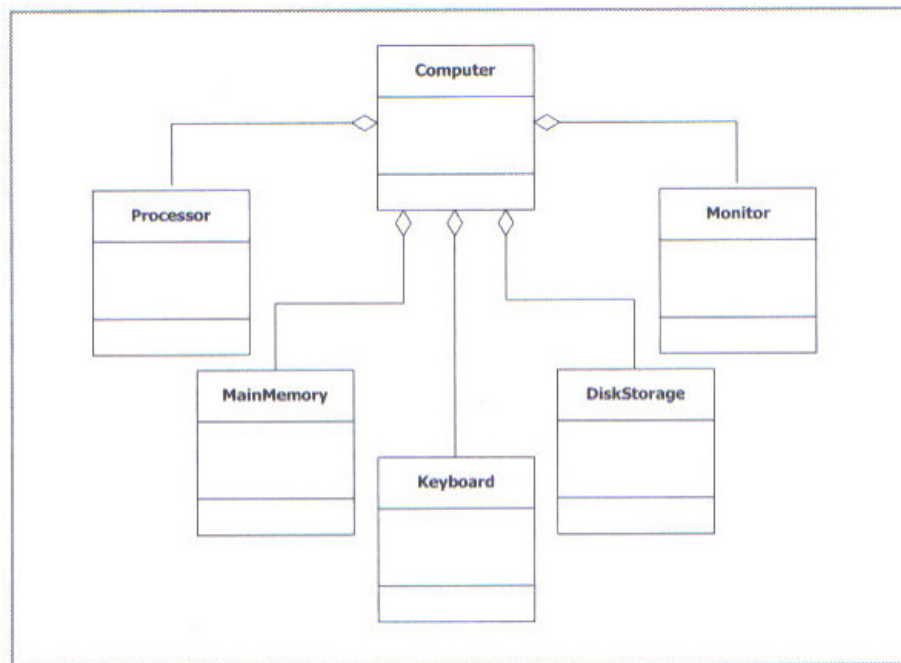
ความสัมพันธ์ของออบเจกต์ และการมีส่วนร่วม (Object Relationships and Associations)

2. *Aggregation* เป็นความสัมพันธ์แบบเป็นส่วนหนึ่งของ ซึ่งมักเรียกว่าความสัมพันธ์แบบ “Whole-Part” หรือ “is part of” โดยจะมีคลาสที่ใหญ่ที่สุดที่เป็นออบเจกต์หลัก และมีคลาสหรือออบเจกต์ส่วนอื่น ๆ เป็นส่วนหนึ่งของออบเจกต์หลัก



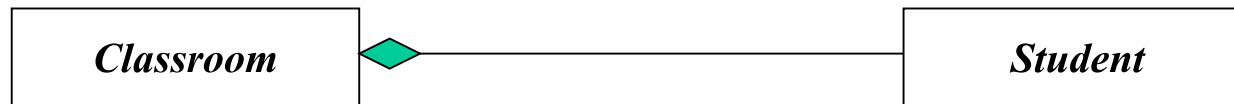
ความสัมพันธ์ของออบเจกต์ และการมีส่วนร่วม (Object Relationships and Associations)

2. *Aggregation* เป็นความสัมพันธ์แบบเป็นส่วนหนึ่งของ ซึ่งมักเรียกว่าความสัมพันธ์แบบ “Whole-Part” หรือ “is part of” โดยจะมีคลาสที่ใหญ่ที่สุดที่เป็นออบเจกต์หลัก และมีคลาสหรือออบเจกต์ส่วนอื่น ๆ เป็นส่วนหนึ่งของออบเจกต์หลัก



ความสัมพันธ์ของออบเจกต์ และการมีส่วนร่วม (Object Relationships and Associations)

3. *Composition* เป็นความสัมพันธ์แบบขึ้นต่อกัน และมีความเกี่ยวข้องกันเสมอ

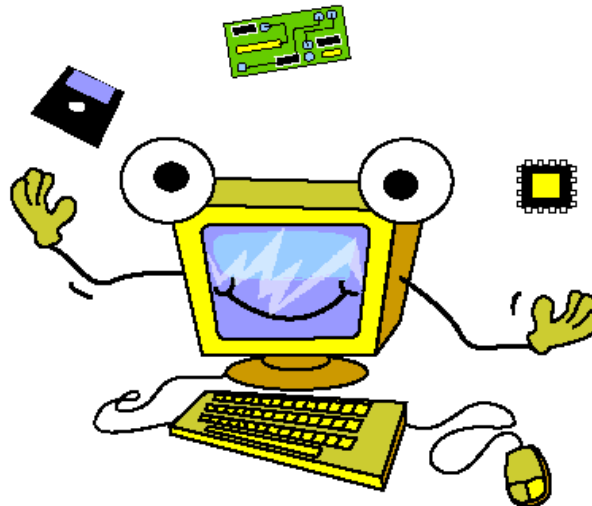


แสดงความสัมพันธ์แบบ *Composition* ระหว่างห้องเรียน และนักศึกษา



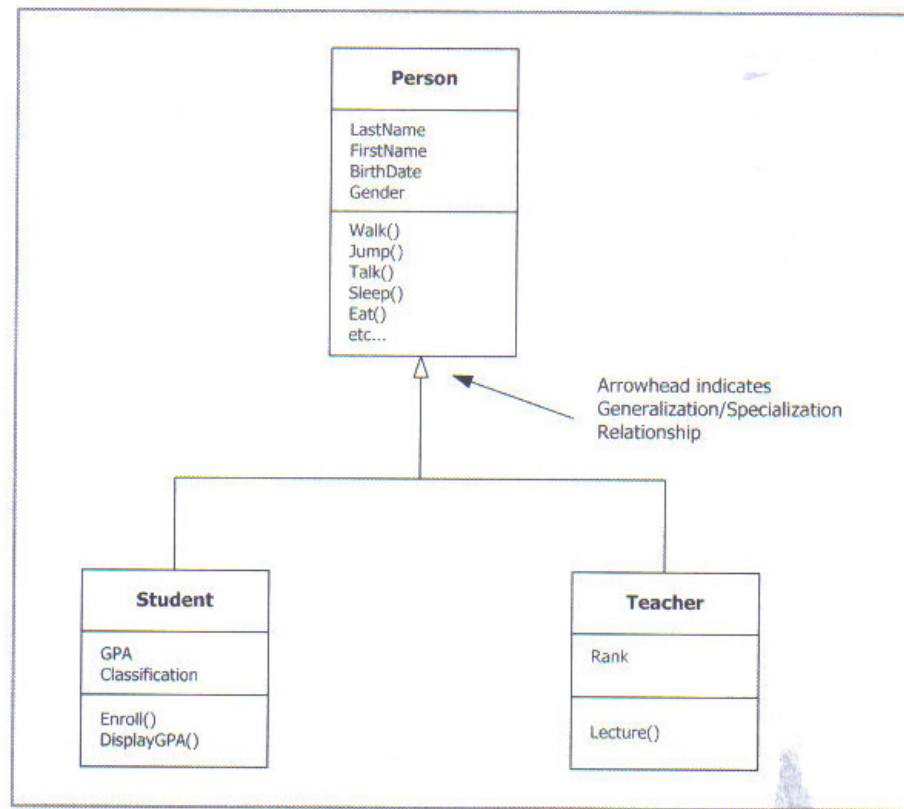
ความสัมพันธ์ของออบเจกต์ และการมีส่วนร่วม (Object Relationships and Associations)

3. *Generalization* เป็นความสัมพันธ์ระหว่างคลาสในลักษณะของการสืบทอดคุณสมบัติจากโครงสร้างคลาสหนึ่งไปยังโครงสร้างอีกคลาสหนึ่ง โดยที่ *Generalization/Specialization* ก็คือเทคนิคการนำคุณสมบัติ และพฤติกรรมของ *Supperclass* ถ่ายทอดคุณสมบัติไปยังออบเจกต์คลาสหรือ *Subclass*



ความสัมพันธ์ของออบเจกต์ และการมีส่วนร่วม (Object Relationships and Associations)

3. Generalization



หลักการพัฒนาระบบเชิงวัตถุ

หลักการจัดแบ่งประเภทของวัตถุในลักษณะทางนามธรรม (Abstract) ออกเป็นกลุ่ม ๆ ที่เรียกว่าคลาส (Class) แต่ละคลาสก็จะมีสถานะ (States) รวมทั้งพฤติกรรม (Behavior) ตามบทบาทของตน โดยมีข้อมูลรายละเอียดหรือคุณสมบัติ (Characteristic) ที่เก็บซ่อน (Encapsulate) ในคลาสของตน และไม่มีการปะปนกับคลาสอื่น ๆ แต่ในด้านการติดต่อสื่อสารหรือการร้องขอใช้บริการ จะสามารถติดต่อสื่อสารกันได้ด้วยข่าวสารหรือเมสเสจ (Message)



หลักการพัฒนาระบบเชิงวัตถุ

OOADI (Object-Oriented Analysis, Design and Implementation)

OOA (Object-Oriented Analysis) คือ วิธีการวิเคราะห์ถึงความต้องการระบบ จากรายละเอียดของคลาส (Class) และวัตถุ (Object) ที่ค้นพบได้จากปัญหาที่เรามุ่งสนใจ เพื่อทำความเข้าใจในรายละเอียดของปัญหาเหล่านั้น

OOD (Object-Oriented Design) คือ วิธีการออกแบบกระบวนการ ด้วยการสร้างแบบจำลองเชิงวัตถุที่สามารถแสดงความหมาย (Notation) ออกมาในรูปแบบเชิงลอจิกัล (Class and Object) และฟลิต์ลจองระบบ

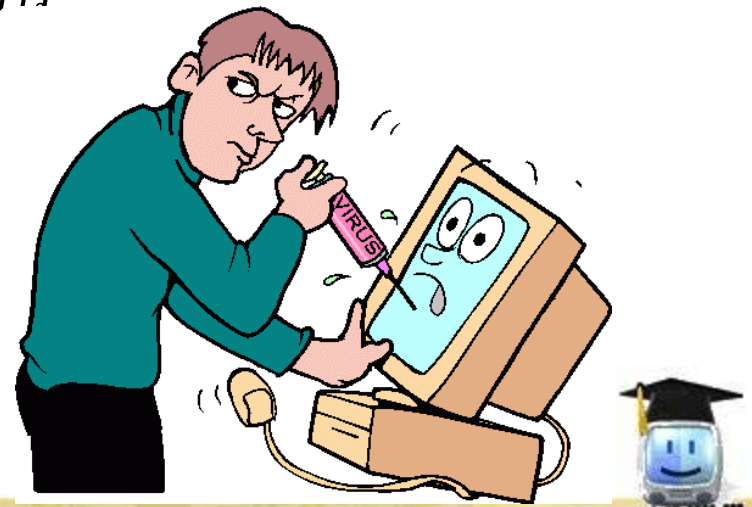
OOI (Object-Oriented Implementation) คือ วิธีการสร้างโปรแกรมเพื่อนำไปใช้งานให้เกิดผล ด้วยการจัดการกลุ่มของวัตถุต่าง ๆ ให้ทำงานร่วมกัน ซึ่งอาจเรียกว่า OOP (Object-Oriented Programming)



UML (Unified Modeling Language)

UML เป็นสัญลักษณ์ (Notation) ที่ใช้อธิบาย แสดงรายละเอียด จำลองการสร้าง และจัดการกับเอกสารต่าง ๆ ในระบบ เพื่อให้การออกแบบซอฟต์แวร์สามารถทำได้ โดยง่าย และปรับปรุงวิธีการทำงานให้ดีขึ้น

UML Diagram ประกอบไปด้วยแบบจำลองทางสถาปัตยกรรมของระบบใน มุมมองต่าง ๆ การพัฒนาระบบงานอาจไม่จำเป็นต้องใช้ทุกไดอะแกรมก็ได้ อาจ พิจารณาเพียงไดอะแกรมที่เหมาะสมต่อความต้องการ



UML (Unified Modeling Language)

UML ประกอบด้วย

- 1. Use Case Diagram*
- 2. Class Diagram (Static)*
- 3. Behavior Diagram (Dynamic):*

3.1 Iteration Diagram

3.1.1 Sequence Diagram

3.1.2 Collaboration Diagram

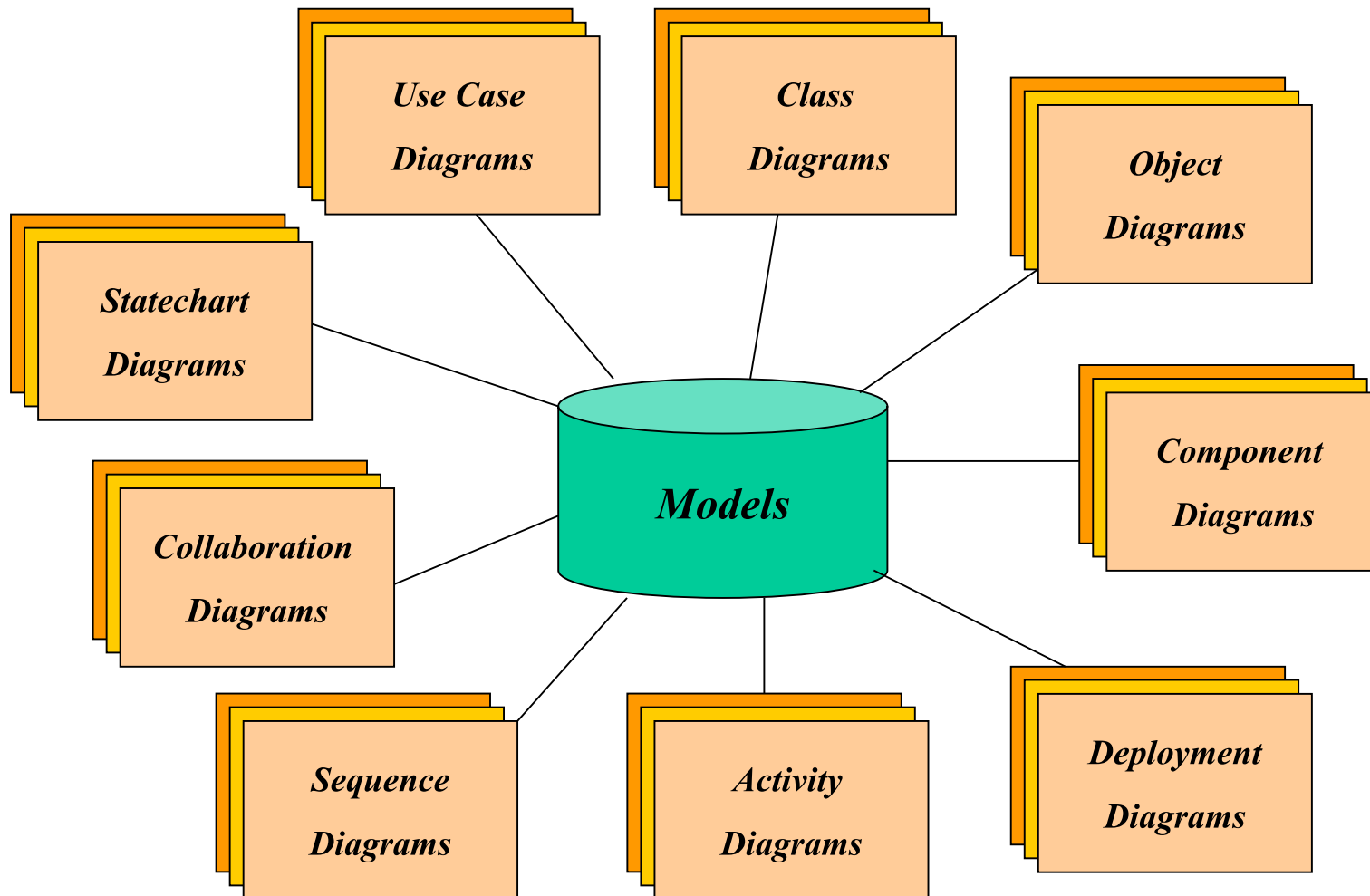
3.2 Statechart Diagram

3.3 Activity Diagram

- 4. Implementation Diagram:*



UML (Unified Modeling Language)



Use Case

เป็นการบ่งบอก และเน้นผู้ใช้งานว่าต้องการทำอะไรในระบบ เป็นการพิจารณาจากมุมมองของผู้ใช้งานที่มีต่อระบบ *Use Case Diagram* ประกอบด้วย

Actor มีสัญลักษณ์เป็นรูปคน หมายถึงผู้เกี่ยวข้องที่ใช้งานระบบ องค์ประกอบที่แสดงเอ็นติตี้ที่อยู่ภายนอกระบบ และมีปฏิสัมพันธ์กับระบบ รวมถึงแสดงความสัมพันธ์กับ *Use Case*

Use Case ใช้สัญลักษณ์รูปวงรี ที่แสดงถึงฟังก์ชันหน้าที่ต่าง ๆ ในระบบ หรือสิ่งที่ระบบต้องทำในมุมมองของผู้ใช้งาน

Relationship แสดงความสัมพันธ์ระหว่าง *Use Case*, *Use Case* กับ *Actor* หรือ *Actor* กับ *Actor* โดยความสัมพันธ์ก็จะจะเป็นไปตามความสัมพันธ์ในรูปแบบต่าง ๆ เช่น *Association*, *Aggregation* หรือ *Composition* และ *Generalization*



Use Case



Actor คือบุคคล หน่วยงาน ระบบงานที่อยู่ภายนอก

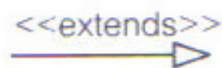


Use Case คืองานต่าง ๆ

Relationships



ความสัมพันธ์ระหว่าง Actor กับ Use Case



ความสัมพันธ์ระหว่าง Use Case กับ Use Case



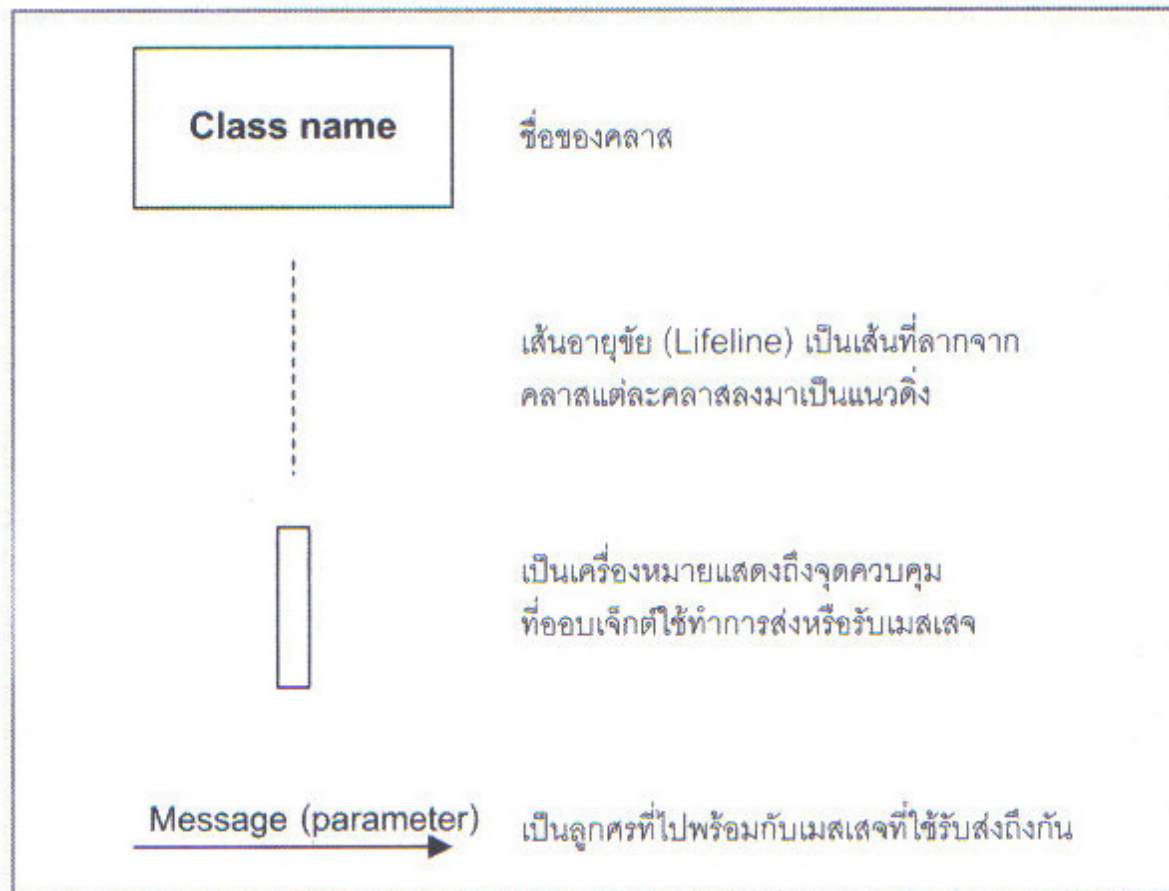
Sequence Diagram

เป็นไคอะแกรมที่ใช้อธิบายการทำงานของ *Use Case* เพื่อแสดงถึงขั้นตอนการทำงานและแสดงลำดับของเมสเสจที่ส่งผ่านระหว่างคลาสที่โต้ตอบกัน นอกจากนี้แล้ว *Sequence Diagram* ยังรวมถึงเงื่อนไขเวลาที่ใช้ในการทำงานด้วย

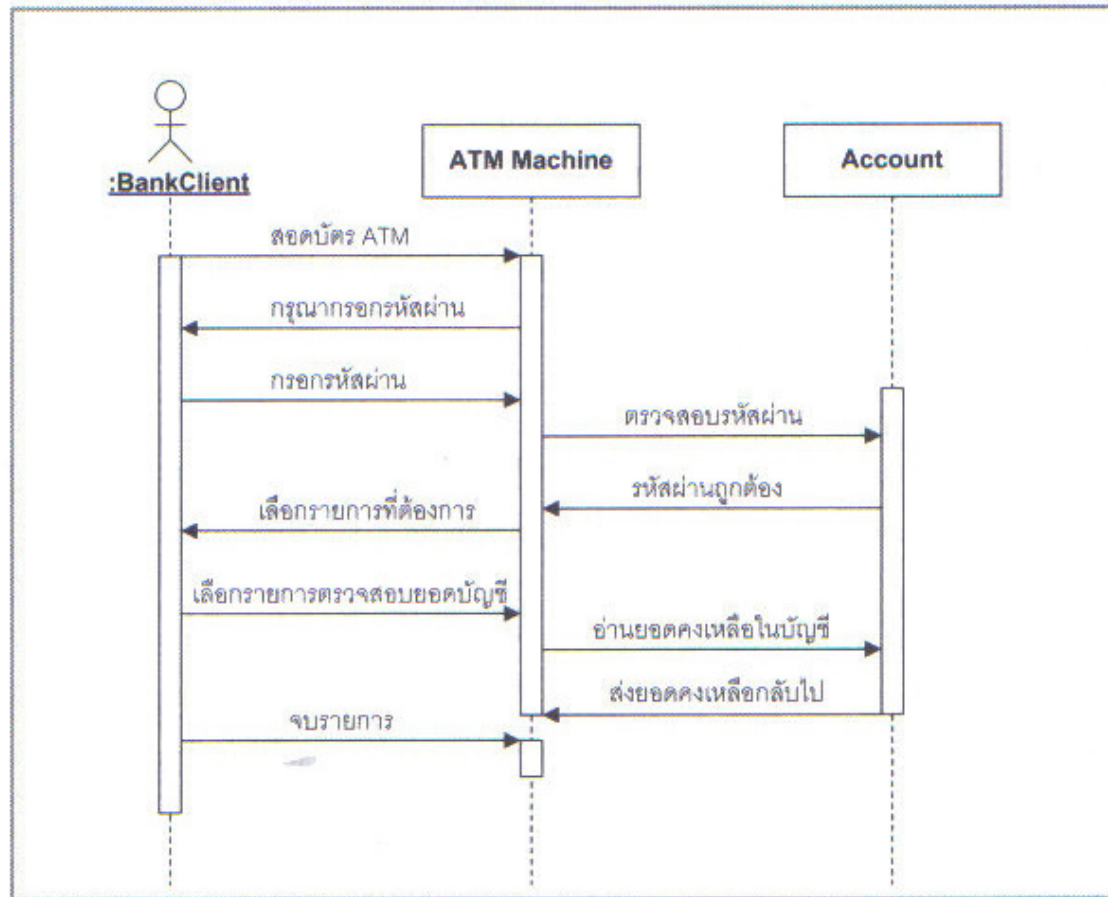
Sequence Diagram จะแสดงในรูปแบบ 2 มิติ โดยเส้นแนวตั้ง (*Vertical*) จะนำเสนอในด้านเวลา และเส้นแนวนอน (*Horizontal*) จะนำเสนอเกี่ยวกับการโต้ตอบระหว่างออบเจกต์หรือคลาสต่าง ๆ เส้นแนวตั้งหรือแนวดิ่งที่เป็นเส้นประนี้จะเรียกว่าเส้นอายุขัย (*Lifeline*) ที่ใช้แสดงช่วงเวลาตั้งแต่เริ่มถูกสร้างจนกระทั่งถูกทำลายที่ใช้แต่ละคลาสโต้ตอบกัน โดยสัญลักษณ์ต่าง ๆ ที่ใช้ใน *Sequence Diagram*



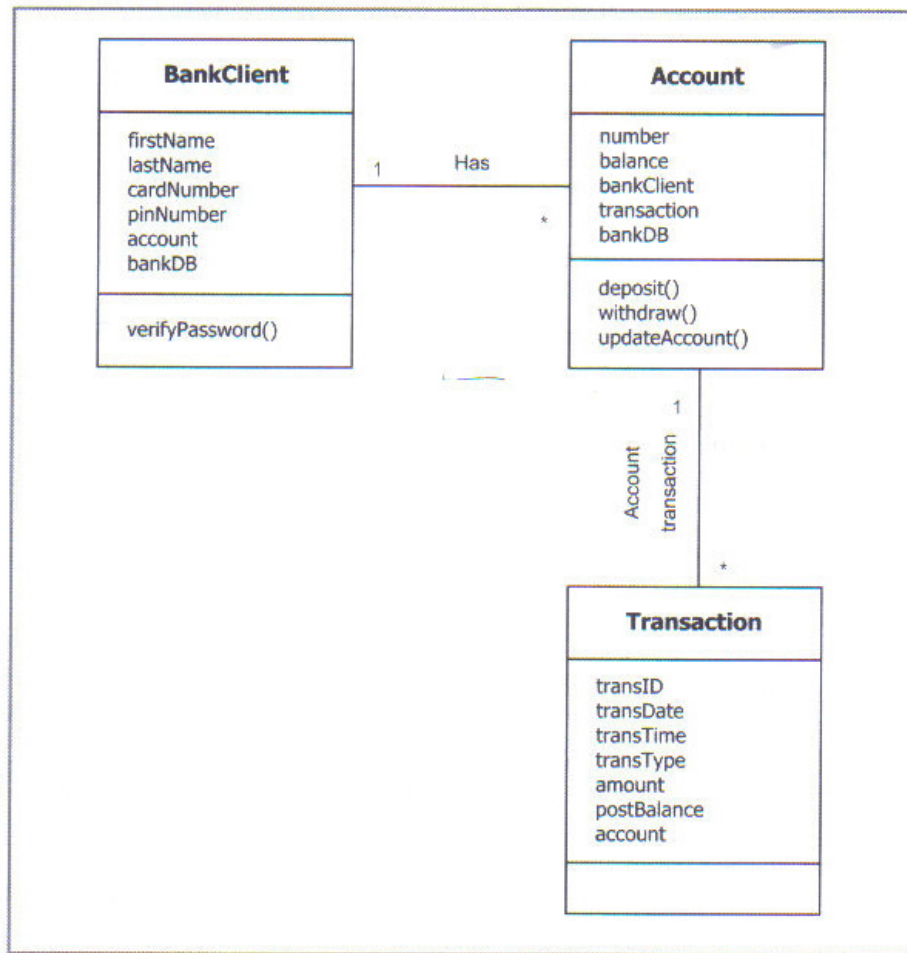
Sequence Diagram



Sequence Diagram



Class Diagram



จะประกอบด้วย
คลาสต่าง ๆ และ
ความสัมพันธ์
ระหว่างคลาส โดย
แต่ละคลาสจะ
แสดงองค์ประกอบ
ที่มีในระบบ และมี
ความสัมพันธ์
(Relationship) ใน
ลักษณะต่าง ๆ

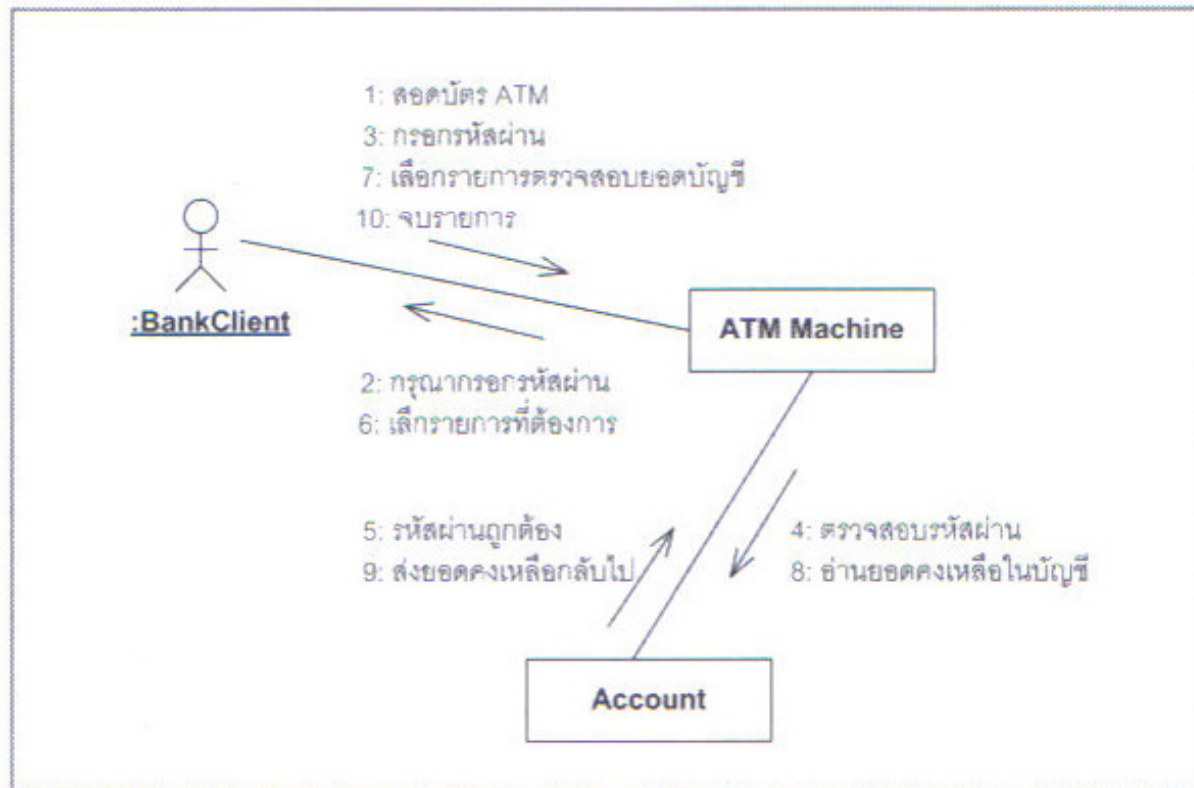


Collaboration Diagram

เป็นไคอะแกรมชนิดเดียวกัน *Sequence Diagram* โดย *Sequence Diagram* จะเป็นไคอะแกรมที่แสดงถึงการแลกเปลี่ยนข่าวสาร แต่ *Collaboration Diagram* จะนำเสนอแผนภาพการทำงานร่วมกันระหว่างออบเจกต์เป็นสำคัญ นอกจากนี้ก็ยังแสดงลำดับการทำงานก่อนและหลังด้วย ซึ่งจะเห็นได้ว่า *Collaboration Diagram* จะแสดงให้เห็นภาพโครงสร้างระบบมากกว่าการเน้นเพียงข่าวสารที่สื่อสารกัน หากต้องการแผนภาพที่มุ่งเน้นด้านเวลาเป็นสำคัญ และแสดงลำดับก่อนหลัง ให้เลือกใช้ *Sequence Diagram* แต่หากต้องการแผนภาพที่ให้ความสัมพันธ์ภายในออบเจกต์ ก็ให้เลือกใช้ *Collaboration Diagram*

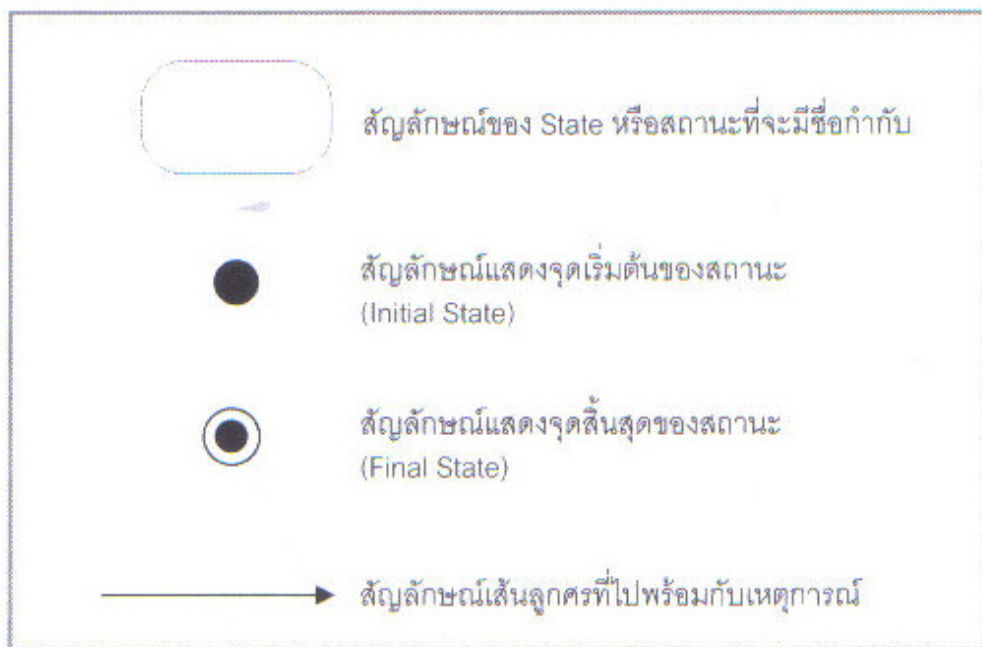


Collaboration Diagram

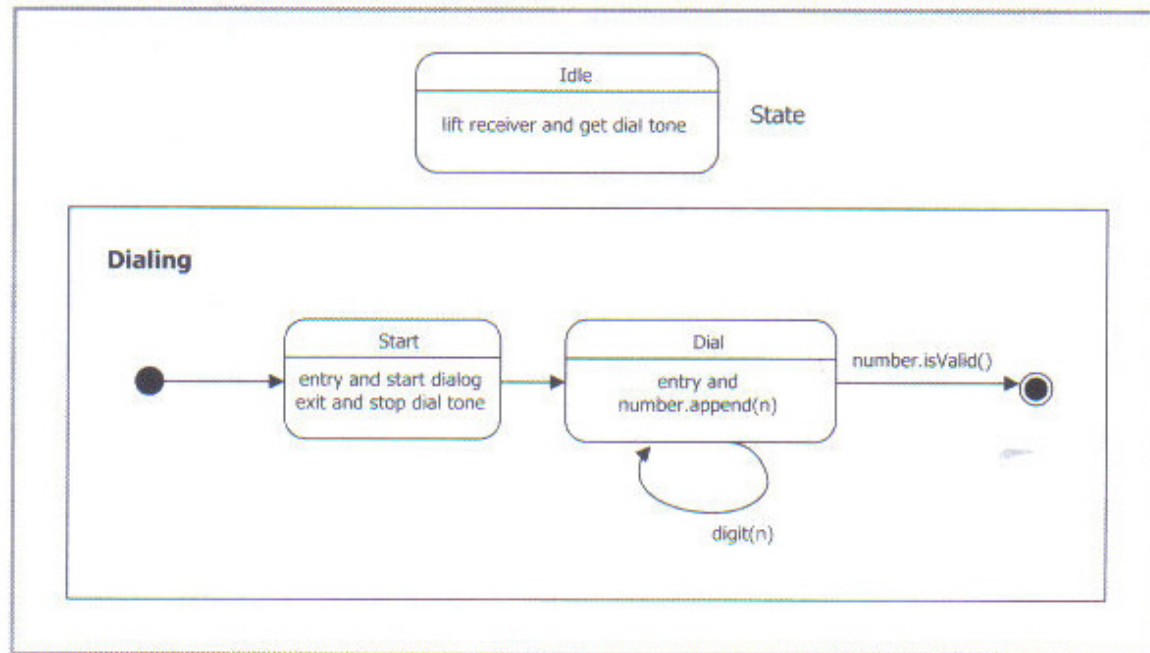


Statechart Diagram

เป็นไคอะแกรมที่แสดงเหตุการณ์ต่าง ๆ ของแต่ละ State ที่มีผลทำให้สถานะของออบเจกต์เปลี่ยนแปลง และผลจากการกระทำที่เกิดขึ้นเมื่อสถานะของออบเจกต์นั้นเปลี่ยน โดยสัญลักษณ์ต่าง ๆ ที่ใช้ใน Statechart Diagram

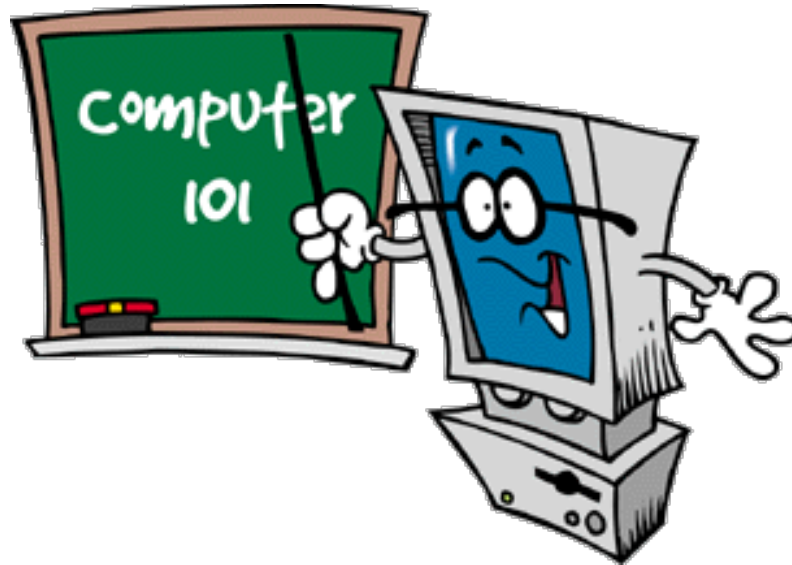


Statechart Diagram

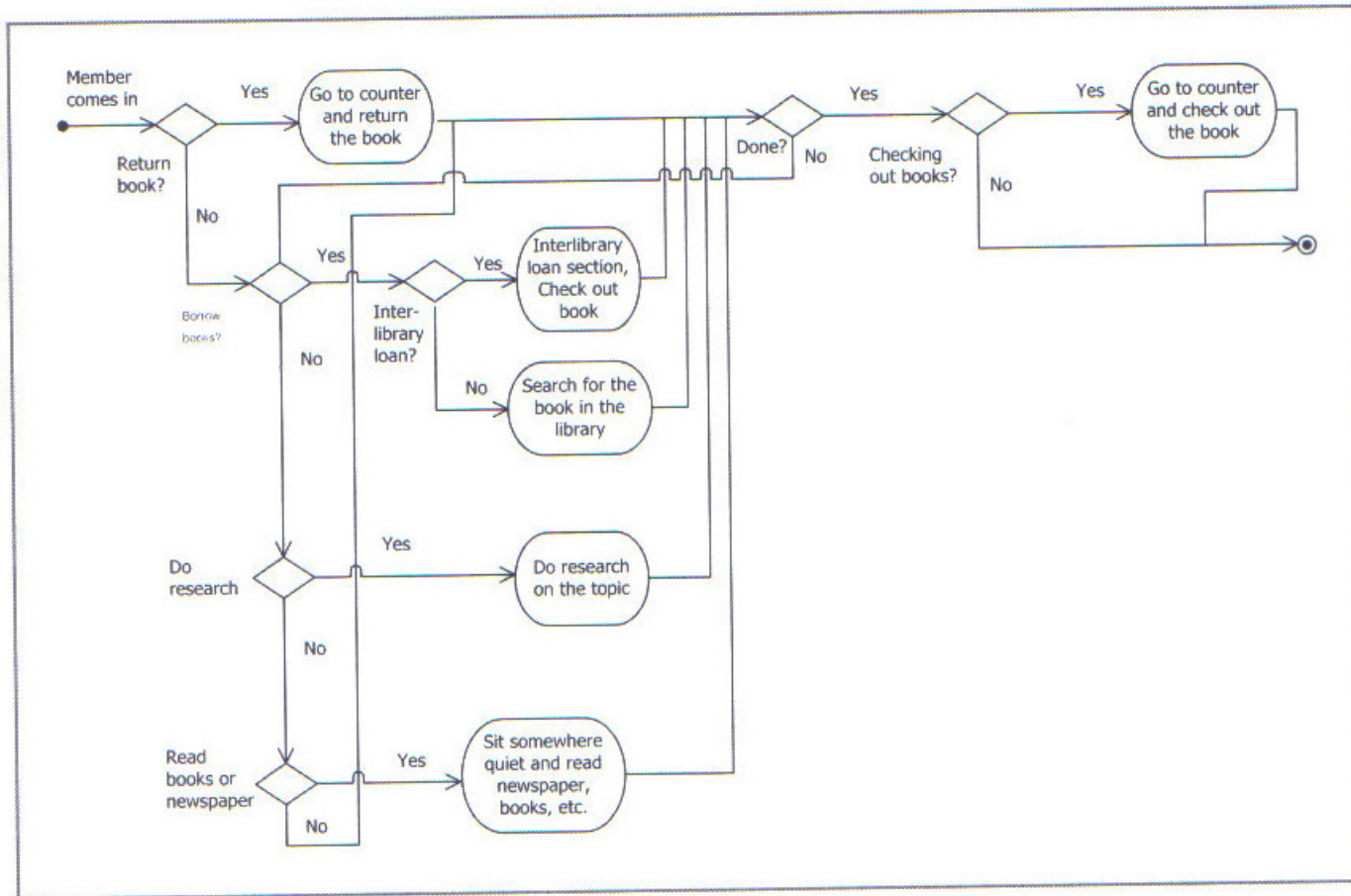


Activity Diagram

เป็นไคอะแกรมแสดงขั้นตอนของการปฏิบัติงาน หรือกิจกรรมในการปฏิบัติงาน โดยจะเกิดสถานะ (State) ต่าง ๆ ที่เกิดขึ้นในระหว่างการทำงาน และผลจากการทำงาน และผลจากการทำงานในขั้นตอนต่าง ๆ ในระบบ

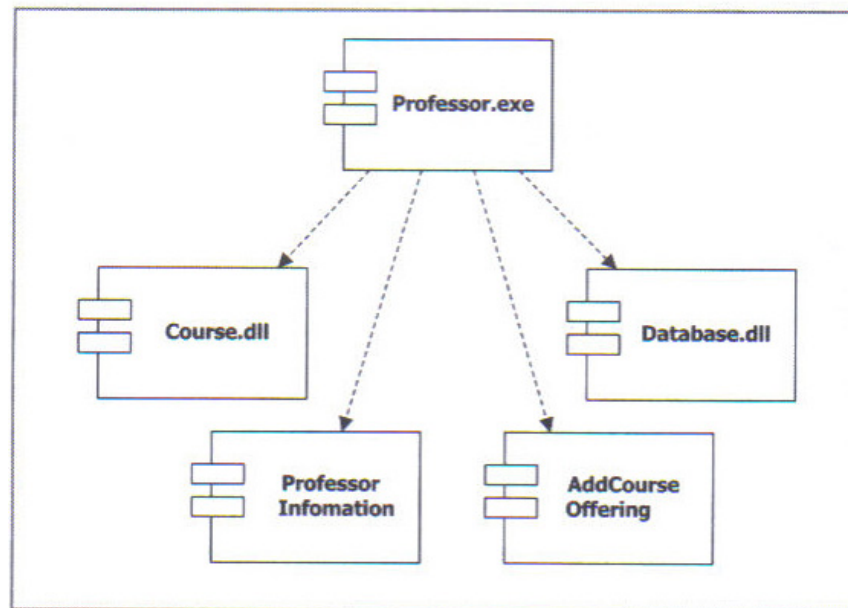


Activity Diagram



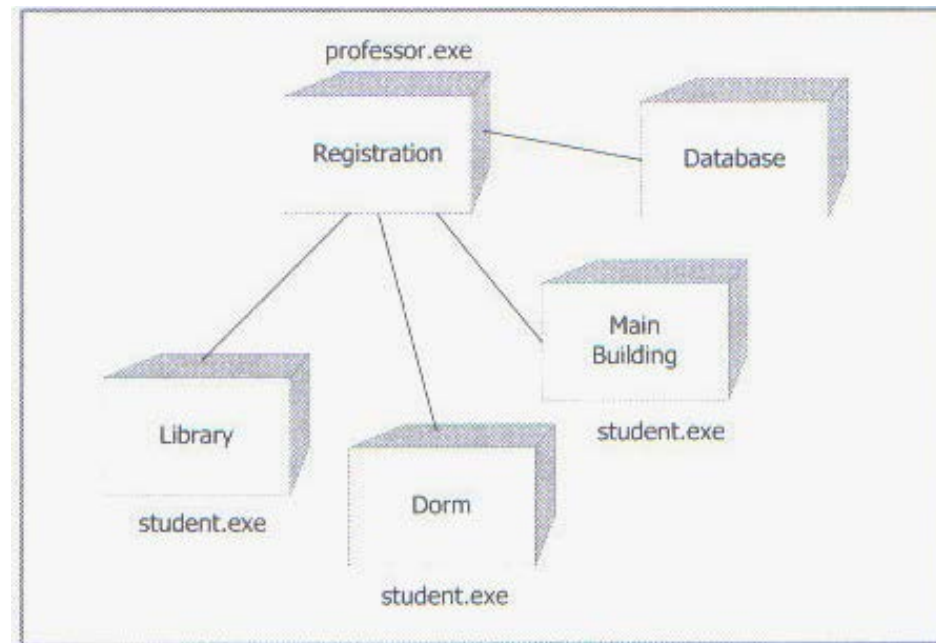
Component Diagram

เป็นไดอะแกรมแสดงโครงสร้างทางกายภาพ และความสัมพันธ์ระหว่างองค์ประกอบต่าง ๆ ของซอฟต์แวร์ ชุดคำสั่ง (Source Code) โปรแกรมที่สามารถเอ็กซ์คิวต์ได้ตัวเอง (Executable Program) โปรแกรมแบบไบนารี (Binary) รวมถึงข้อความ (Text) และยูสเซอร์อินเตอร์เฟซ



Deployment Diagram

เป็นไดอะแกรมที่แสดงที่ตั้ง ของส่วนประสมผล รวมทั้ง *Software Component* ต่าง ๆ ซึ่งอาจกล่าวได้ว่า *Deployment Diagram* เป็นแผนภาพแสดงสถาปัตยกรรมของ ฮาร์ดแวร์และซอฟต์แวร์ในระบบ รวมถึงความสัมพันธ์ระหว่างฮาร์ดแวร์และซอฟต์แวร์





The End